

Sql_Pyspark_Commands

Task	SQL Command	PySpark Command
Selecting Data	SELECT column1, column2 FROM table_name	df.select("column1", "column2")
Filtering Data	SELECT * FROM table_name WHERE condition;	df.filter(condition)
Grouping and Aggregating	SELECT column1, COUNT(*) FROM table_name GROUP BY column1;	df.groupBy("column1").count()
Joining Data	SELECT * FROM table1 JOIN table2 ON table1.column = table2.column;	df1.join(df2, df1.column == df2.column)
Ordering Data	SELECT * FROM table_name ORDER BY column ASC;	df.orderBy("column", ascending=True)
Creating Temporary Views	CREATE OR REPLACE TEMP VIEW view_name AS SELECT * FROM table_name;	df.createOrReplaceTempView("view_name")
Running SQL Queries	SELECT * FROM view_name WHERE condition;	spark.sql("SELECT * FROM view_name WHERE condition").show()
Writing Data to Table	INSERT INTO table_name (column1, column2) VALUES (value1, value2);	df.write.mode("overwrite").saveAsTable("table_name")
Reading Data from Table	SELECT * FROM table_name;	df = spark.read.table("table_name")
Counting Rows	SELECT COUNT(*) FROM table_name;	df.count()
Dropping Duplicates	SELECT DISTINCT column1, column2 FROM table_name;	df.dropDuplicates(["column1", "column2"])
Aggregating by Condition	SELECT column1, AVG(column2) FROM table_name GROUP BY column1;	df.groupBy("column1").avg("column2")
Renaming Columns	SELECT column1 AS new_column FROM table_name;	df.withColumnRenamed("column1", "new_column")
Limiting Rows	SELECT * FROM table_name LIMIT 10;	df.limit(10)
Calculating Sum	SELECT SUM(column1) FROM table_name;	df.selectExpr("sum(column1)").show()
Dropping Columns	SELECT column2 FROM table_name;	df.drop("column1")
Handling Null or Not Null Values	SELECT * FROM table_name WHERE column IS NULL;	df.filter(df.column.isNull())
Greater Than or Equal	SELECT * FROM table_name WHERE column >= value;	df.filter(df.column >= value)
Between	SELECT * FROM table_name WHERE column BETWEEN value1 AND value2;	df.filter(df.column.between(value1, value2))
LIKE	SELECT * FROM table_name WHERE column LIKE 'pattern';	df.filter(df.column.like("pattern"))
CAST	SELECT CAST(column AS data_type) FROM table_name;	df.selectExpr("CAST(column AS data_type)").show()

String_Operations

Task	SQL Command	PySpark Command
Upper Case	SELECT UPPER(column) FROM table_name;	df.selectExpr("UPPER(column)").show()
Lower Case	SELECT LOWER(column) FROM table_name;	df.selectExpr("LOWER(column)").show()
Length	SELECT LENGTH(column) FROM table_name;	df.selectExpr("LENGTH(column)").show()
Substring	SELECT SUBSTRING(column, start, length) FROM table_name;	df.selectExpr("SUBSTRING(column, start, length)").show()
Concatenation	SELECT CONCAT(column1, column2) FROM table_name;	df.selectExpr("CONCAT(column1, column2)").show()
Trim	SELECT TRIM(column) FROM table_name;	df.selectExpr("TRIM(column)").show()
Replace	SELECT REPLACE(column, 'old_string', 'new_string') FROM table_name;	df.selectExpr("REPLACE(column, 'old_string', 'new_string')").show()
Instr	SELECT INSTR(column, 'search_string') FROM table_name;	df.selectExpr("INSTR(column, 'search_string')").show()
Left Padding	SELECT LPAD(column, length, 'pad_string') FROM table_name;	df.selectExpr("LPAD(column, length, 'pad_string')").show()
Right Padding	SELECT RPAD(column, length, 'pad_string') FROM table_name;	df.selectExpr("RPAD(column, length, 'pad_string')").show()

Date/Time_Operations

Task	SQL Command	PySpark Command
Date Difference	SELECT DATEDIFF(end_date, start_date) FROM table_name;	df.selectExpr("DATEDIFF(end_date, start_date)").show()
Current Date	SELECT CURRENT_DATE;	df.selectExpr("CURRENT_DATE").show()
Current Timestamp	SELECT CURRENT_TIMESTAMP;	df.selectExpr("CURRENT_TIMESTAMP").show()
Date Add	SELECT DATE_ADD(start_date, INTERVAL num_days DAY) FROM table_name;	df.selectExpr("DATE_ADD(start_date, INTERVAL num_days DAY)").show()
Date Subtract	SELECT DATE_SUB(start_date, INTERVAL num_days DAY) FROM table_name;	df.selectExpr("DATE_SUB(start_date, INTERVAL num_days DAY)").show()
Date Format	SELECT DATE_FORMAT(date, 'format') FROM table_name;	df.selectExpr("DATE_FORMAT(date, 'format')").show()

WINDOW_FUNCTIONS

Task	SQL Command	PySpark Command
Sum	SELECT SUM(column) OVER (PARTITION BY partition_column) FROM table_name;	window = Window.partitionBy("partition_column") df.withColumn("new_column", F.sum("column").over(window)).show()
Average	SELECT AVG(column) OVER (PARTITION BY partition_column ORDER BY order_column) FROM table_name;	window = Window.partitionBy("partition_column").orderBy("order_column") df.withColumn("new_column", F.avg("column").over(window)).show()
Row Number	SELECT ROW_NUMBER() OVER (PARTITION BY partition_column ORDER BY order_column) FROM table_name;	window = Window.partitionBy("partition_column").orderBy("order_column") df.withColumn("row_number", F.row_number().over(window)).show()
Rank	SELECT RANK() OVER (PARTITION BY partition_column ORDER BY order_column) FROM table_name;	window = Window.partitionBy("partition_column").orderBy("order_column") df.withColumn("rank", F.rank().over(window)).show()
Dense Rank	SELECT DENSE_RANK() OVER (PARTITION BY partition_column ORDER BY order_column) FROM table_name;	window = Window.partitionBy("partition_column").orderBy("order_column") df.withColumn("dense_rank", F.dense_rank().over(window)).show()